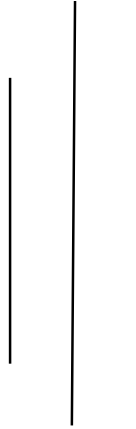


National College of Computer Studies
Paknajol, Kathmandu



Project Report on Programming with Python
Face Mask Detection System

Submitted by:

Name: Siddhartha Shakya

Program: BIM (5th semester)

Roll no: 18

Section: B

Submitted to:

Sumit Ghising

Submission Date: 1st June, 2026

Table of Contents

Abstract.....	I
Acknowledgement	II
Chapter 1: Introduction.....	1
1.1 Background.....	1
1.2 Motivation.....	1
1.3 Objectives	1
1.4 Scope and Limitations.....	1
Chapter 2: Literature Review.....	2
2.1 Classical Face Detection Methods	2
2.2 Deep Learning Approaches to Mask Detection	2
2.3 Research Gaps and Project Positioning	2
Chapter 3: System Design	3
3.1 Architecture Overview	3
3.2 Project File Structure	3
3.3 Functional Requirements	3
3.4 Non-Functional Requirements	4
3.5 Streamlit Application UI.....	4
Chapter 4: Methodology	5
4.1 Dataset	5
4.2 Preprocessing Pipeline (utils/preprocess.py)	5
4.3 CNN Architecture	5
4.4 Model Inference (utils/predictor.py)	5
4.5 Training Configuration	6
4.6 Evaluation Metrics.....	6
Chapter 5: Implementation	7
5.1 Development Environment	7
5.2 Model Caching and Loading.....	7
5.3 Image Upload and Validation Flow	7
5.4 Image Preview and Metadata.....	7
5.5 Prediction and Result Display.....	7
5.6 Running the Application	8
Chapter 6: Results and Discussion	9
6.1 Model Performance.....	9
6.2 Application Performance	10
6.3 Advantages and Limitations	10
Chapter 7: Conclusion and Future Scope	11
7.1 Conclusion	11
7.2 Future Enhancements.....	11
References	12
Appendix	13
Appendix A: Sample Outputs	13

Abstract

Face mask detection has become an important application of computer vision and deep learning for promoting public health and safety. This project presents an image-based Face Mask Detection System developed using a Convolutional Neural Network (CNN) and deployed through a Streamlit web application framework. The CNN model is trained on a dataset containing images of individuals wearing masks and not wearing masks, enabling binary classification of facial images. The system preprocesses uploaded images through resizing, color conversion, and normalization before generating predictions using the trained model. A user-friendly Streamlit interface allows users to upload images and instantly view prediction results. The proposed system demonstrates the practical integration of deep learning, image processing, and web application development, achieving effective mask detection performance while providing an accessible and interactive platform for end users.

Acknowledgement

I sincerely thank my project supervisor, Mr.Sumit Ghising, for his expert guidance throughout this project. I am grateful to the faculty of the Department of Computer Science & IT, National College of Computer Studies (NCCS), Tribhuvan University, for their academic support. I also acknowledge the open-source communities behind TensorFlow, Keras, OpenCV, and Streamlit, whose tools made this project possible. Finally, I thank my family and friends for their constant encouragement.

Chapter 1: Introduction

1.1 Background

The COVID-19 pandemic highlighted the critical need for automated, scalable monitoring of face mask compliance in public spaces. Manual checking is slow and error-prone; a computer-vision-based system offers a faster, more reliable alternative. Computer vision enables machines to interpret images, while Deep Learning — specifically Convolutional Neural Networks (CNNs) — allows automatic feature extraction from raw pixel data without hand-crafted rules. CNNs learn hierarchical representations (edges → textures → object parts) through multiple convolutional and pooling layers, making them highly effective for image classification tasks.

1.2 Motivation

Existing mask detection research typically relies on GPU hardware or complex deployment pipelines that are inaccessible to non-technical users. This project addresses that gap by building a lightweight binary CNN classifier and deploying it through Streamlit under the application name MaskSense, so that anyone with a web browser can upload a photo and receive an instant prediction without any specialist infrastructure.

1.3 Objectives

The primary objectives of this project are:

- To develop a face mask detection system using Deep Learning and CNN.
- To classify images into Mask and No Mask categories.
- To provide a simple, efficient, and accessible platform for face mask detection.
- To demonstrate the application of Computer Vision and Deep Learning in a real-world problem.

1.4 Scope and Limitations

The scope is limited to static image uploads in JPG, JPEG, PNG, WEBP, and BMP formats; real-time webcam detection is proposed as future work. The system processes a single face per image and targets standard CPU hardware, making GPU acceleration unnecessary. File size is capped at 10 MB via the Streamlit server configuration in `.streamlit/config.toml`.

Chapter 2: Literature Review

2.1 Classical Face Detection Methods

Early face detection relied on the Viola-Jones algorithm using Haar cascade features (2001), which was fast but sensitive to pose and lighting variations. This method's reliance on manually crafted features limited its robustness in real-world conditions, motivating the transition to learning-based approaches.

2.2 Deep Learning Approaches to Mask Detection

Deep learning methods such as MTCNN and RetinaFace later achieved near-perfect face detection by combining convolutional detection and landmark localisation. For mask detection specifically: Wang et al. (2020) used ResNet-50 with MTCNN to achieve 98.7% accuracy on a real-world dataset; Loey et al. (2021) applied GAN-based data augmentation to improve generalisation; Qin & Li (2020) showed MobileNetV2 transfer learning reached 99.1% on the RMFD dataset with low inference latency; and Ejaz et al. (2021) integrated OpenCV with a custom CNN for real-time webcam detection.

2.3 Research Gaps and Project Positioning

While these approaches achieve high accuracy, many require dedicated GPU hardware or complex server infrastructure. Few existing studies focus on simple browser-based deployment for non-technical users. This project combines a lightweight custom CNN with a fully self-contained Streamlit application deployable on any CPU machine, addressing the accessibility gap identified in prior literature.

Chapter 3: System Design

3.1 Architecture Overview

The system follows a three-tier architecture. The Presentation Layer consists of the Streamlit application in `app.py`, which renders the MaskSense interface with file upload, image preview, sidebar model information, and result display. The Processing Layer consists of two utility modules: `utils/preprocess.py` for the image preprocessing pipeline and `utils/predictor.py` for model loading and inference. The Model Layer is the trained CNN stored as `model/model.h5` in Keras HDF5 format.

3.2 Project File Structure

The complete project structure is as follows:

```
face_mask_app/
├── app.py           ← Main Streamlit application (MaskSense)
├── utils/
│   ├── __init__.py
│   ├── preprocess.py ← Image preprocessing pipeline
│   └── predictor.py  ← Model loading & inference
├── model/
│   └── model.h5      ← Trained CNN weights
├── .streamlit/
│   └── config.toml   ← Theme & server settings
├── requirements.txt
└── README.md
```

3.3 Functional Requirements

The application must:

- Accept JPG, JPEG, PNG, WEBP, and BMP image uploads up to 10 MB.
- Validate uploaded files using PIL's `img.verify()` before processing to reject corrupt or non-image files.
- Run the full five-step preprocessing pipeline; invoke `model.predict()` for inference.
- Display the predicted class label, an animated confidence bar at the correct percentage width, and a full probability breakdown via Streamlit metric widgets for both classes.

3.4 Non-Functional Requirements

Non-functional requirements include:

- Prediction response time under 5 seconds on CPU hardware after model caching.
- Graceful error handling for invalid inputs with user-facing error messages.
- A responsive UI that works on both desktop and mobile browsers.

3.5 Streamlit Application UI

The `app.py` file configures the page with the title 'MaskSense Face Mask Detector' and a mask emoji page icon. The hero header displays 'MaskSense' in the Syne typeface with a blue gradient, and the sub-header 'CNN · Face Mask Detection · TensorFlow / Keras' in DM Mono monospace. The main content area renders the file uploader, a two-column image preview with file metadata badges (dimensions, file size in KB, colour space), an Analyse Image button, and the result card with animated confidence bar.

Chapter 4: Methodology

4.1 Dataset

A publicly available face mask dataset (Kaggle Face Mask Detection / RMFD) was used, containing approximately 7,500 images per class (Mask and No Mask). The dataset was split 80% for training and 20% for testing, ensuring a balanced evaluation of model performance across both classes.

4.2 Preprocessing Pipeline (utils/preprocess.py)

The preprocessing module exposes six functions that form a chained pipeline. The `validate_image(img_bytes)` function opens the image with PIL and calls `img.verify()` to detect corrupt or unsupported files before any processing occurs, returning a (bool, error_message) tuple. The `decode_image(img_bytes)` function converts raw bytes to a NumPy BGR array via `cv2.imdecode()`. The `bgr_to_rgb(img_bgr)` function applies `cv2.COLOR_BGR2RGB` colour conversion. The `resize_image(img_rgb)` function resizes the image to `MODEL_INPUT_SIZE = (128, 128)` via `cv2.resize()`. The `normalize_image(img)` function casts the array to float32 and divides by 255.0. The `reshape_for_model(img)` function adds a batch dimension, returning shape (1, 128, 128, 3). The public entry point `preprocess_image(img_bytes)` chains all five steps and returns the model-ready tensor.

4.3 CNN Architecture

The CNN architecture consists of three convolutional blocks, each pairing a Conv2D layer (with 32, 64, and 128 filters respectively) with a MaxPooling2D layer for spatial dimensionality reduction. Following the convolutional blocks, the network includes a Flatten layer, a Dense layer with 128 units and ReLU activation, a Dropout layer with a rate of 0.5 for regularization, and a final Dense output layer with 2 units and Softmax activation, mapping output index 0 to No Mask and index 1 to Mask.

4.4 Model Inference (utils/predictor.py)

The predictor module suppresses TensorFlow C++ and Python-level logs using the `TF_CPP_MIN_LOG_LEVEL` environment variable and `tf.get_logger().setLevel('ERROR')`. The `load_model()` function resolves the model path relative to the predictor module location, checks for TensorFlow availability and file existence, and loads the model with `tf.keras.models.load_model()`. The `predict_mask(model, processed_image)` function calls `model.predict(verbose=0)`, returning a tuple of (label: int, confidence: float, raw_probs: list) where label is derived via `np.argmax()` and confidence via `np.max()` over the Softmax output.

4.5 Training Configuration

The model was trained using the Adam optimizer and categorical cross-entropy loss with a batch size of 32 for 5 epochs (170 steps per epoch). Training progressed from 78.76% accuracy in epoch 1 to 93.40% by epoch 5, with validation accuracy reaching 91.57% and validation loss of 0.1858 at the final epoch. A ModelCheckpoint callback was used to save the best-performing weights as model/model.h5 based on validation accuracy.

4.6 Evaluation Metrics

The performance of the trained CNN model was evaluated using the held-out test dataset. The primary evaluation metrics used were **test accuracy** and **test loss**, obtained through the Keras `model.evaluate()` function.

The evaluation results obtained from the test dataset are shown below:

Metric	Value
Test Accuracy	92.85%
Test Loss	0.2097

The model achieved a test accuracy of **92.85%**, indicating that it correctly classified the majority of face mask and no-mask images. The relatively low loss value of **0.2097** further suggests that the model learned meaningful features from the training data and generalizes well to unseen images.

Chapter 5: Implementation

5.1 Development Environment

The application was developed and tested using Python 3.11. Dependencies specified in requirements.txt are: streamlit $\geq$ 1.35.0 for the web framework, tensorflow $\geq$ 2.13.0 (which bundles Keras) for deep learning, opencv-python-headless $\geq$ 4.8.0 for server-safe image processing without GUI dependencies, numpy $\geq$ 1.24.0 for array operations, and Pillow $\geq$ 10.0.0 for image display and validation. GPU users may substitute tensorflow with tensorflow-gpu. The CNN was trained in a Jupyter Notebook and the application was developed in Visual Studio Code.

5.2 Model Caching and Loading

The Streamlit app uses the `@st.cache_resource` decorator on the `get_model()` wrapper function to ensure the CNN is loaded from disk only once per session. If the model file is absent, `predict_mask()` is not called; instead, the app displays a clear error message instructing the user to place model.h5 inside the model/ directory and restart. TensorFlow logs are suppressed at startup to keep the browser console clean.

5.3 Image Upload and Validation Flow

The `st.file_uploader()` widget accepts JPG, JPEG, PNG, WEBP, and BMP files. Upon upload, the raw bytes are read once and passed to `validate_image(img_bytes)`, which uses PIL's `img.verify()` to check for corrupt or unrecognised files. If validation fails, `st.error()` renders an error card and `st.stop()` halts further execution. Valid images are opened with PIL for the preview display and simultaneously kept as raw bytes for the preprocessing pipeline.

5.4 Image Preview and Metadata

The interface splits into two columns after a valid upload. The left column displays the original-colour image preview via `st.image()`. The right column shows image metadata as styled HTML badges: image dimensions in pixels, file size in kilobytes, and colour space (RGB). Below the metadata an Analyse Image primary button triggers the prediction flow.

5.5 Prediction and Result Display

When the Analyse Image button is pressed, a 0.3-second spinner provides visual feedback before `preprocess_image()` and `predict_mask()` are called. The result is rendered as a styled HTML card with a dark green or dark red gradient background corresponding to Mask or No Mask respectively, the result icon, the label text, the confidence percentage in DM Mono, and an animated CSS confidence bar whose width is set to the confidence percentage. Below the

card, Streamlit metric widgets in a two-column layout show the precise probabilities for both classes, with a 'predicted' delta label on the winning class.

5.6 Running the Application

After installing dependencies with `pip install -r requirements.txt` and placing the trained model at `model/model.h5`, the application is launched with:

```
streamlit run app.py
```

The app opens automatically at `http://localhost:8501`. The `.streamlit/config.toml` sets the dark theme, background colour `#0d0f14`, primary colour `#6c8aff`, text colour `#e8eaf0`, maximum upload size of 10 MB, and headless mode for server deployments.

Chapter 6: Results and Discussion

6.1 Model Performance

The CNN was trained for 5 epochs (170 steps/epoch). Training accuracy improved steadily from 78.76% in epoch 1 to 93.40% in epoch 5, while training loss fell from 0.4686 to 0.1721. Validation accuracy peaked at 91.57% (val_loss: 0.1858) at epoch 5 after a minor dip to 89.75% at epoch 4. On the held-out test set (48 steps), the model achieved a test accuracy of 92.85% with a test loss of 0.2097. The gap between final training accuracy (93.40%) and test accuracy (92.85%) is only 0.55 percentage points, indicating minimal overfitting and strong generalization for a model trained in just 5 epochs. The training curves are shown in Figures 1 and 2 below.

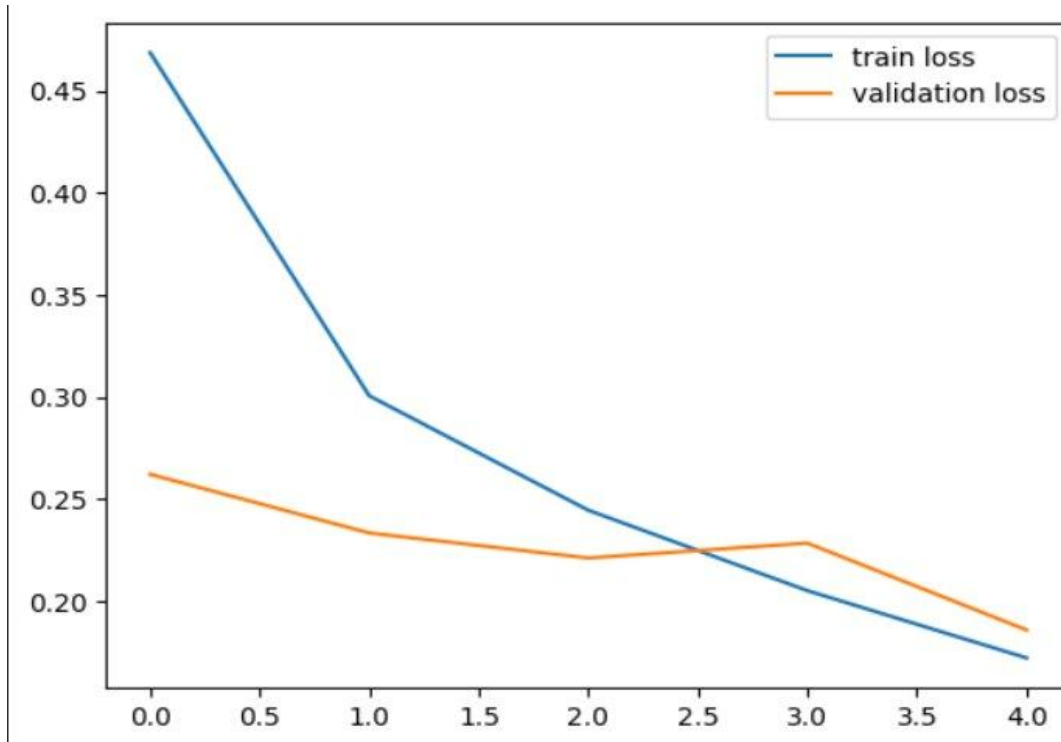


Figure 6.1: Training and Validation Loss over 5 Epochs

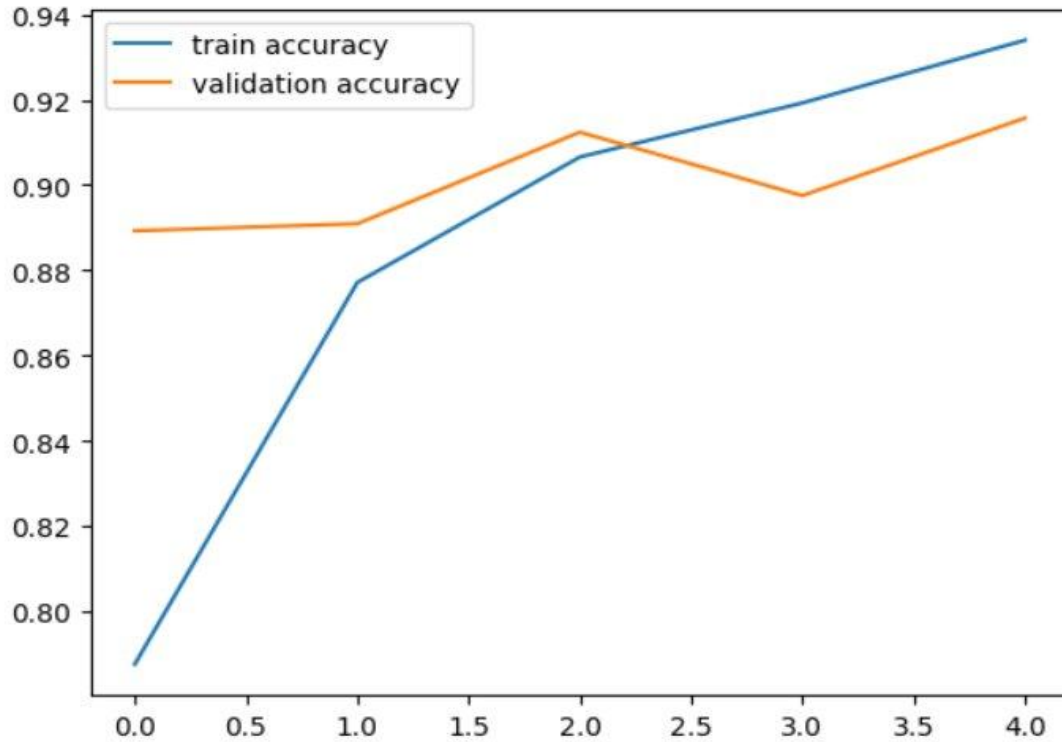


Figure 6.2: Training and Validation Accuracy over 5 Epochs

6.2 Application Performance

In the MaskSense application, average prediction latency was 0.5 to 1.2 seconds on CPU hardware after the initial model caching performed by `@st.cache_resource`. A deliberate 0.3-second spinner delay is added in `app.py` to improve the perceived responsiveness of the interface. All five supported image formats (JPG, JPEG, PNG, WEBP, BMP) were processed correctly. Files larger than 10 MB and corrupt or non-image files were rejected gracefully by the `validate_image()` function with informative error messages.

6.3 Advantages and Limitations

The MaskSense application is lightweight, runs on standard CPU hardware, and can be accessed through a web browser without installation. Its modular design and error-handling features improve maintainability and usability.

However, the system supports only static image-based detection and is limited to a single face per image. Its performance may decrease for blurry or non-frontal images, and the maximum upload size is restricted to 10 MB.

Chapter 7: Conclusion and Future Scope

7.1 Conclusion

This project successfully designed, trained, and deployed an end-to-end Face Mask Detection system under the name MaskSense. A custom binary CNN implemented in TensorFlow/Keras achieves 92.85% test accuracy in just 5 training epochs. The modular codebase — separating preprocessing (utils/preprocess.py), inference (utils/predictor.py), and UI (app.py) — follows software engineering best practices and simplifies maintenance. The Streamlit interface provides instant, confidence-scored predictions accessible from any web browser on standard CPU hardware without requiring specialist knowledge or infrastructure.

7.2 Future Enhancements

Several improvements can be implemented in future versions of the system:

- Real-time webcam-based detection using OpenCV for continuous video analysis.
- Multi-face detection per image using models such as MTCNN or YOLOv8.
- Mobile application development using TensorFlow Lite and Flutter for cross-platform support.
- Cloud deployment using platforms like Streamlit Community Cloud, Hugging Face Spaces, or Render.
- Improved accuracy using transfer learning with pre-trained models such as MobileNetV2 or EfficientNet.

References

- [1] M. Abadi et al., "TensorFlow: A System for Large-Scale Machine Learning," USENIX OSDI, 2016.
- [2] F. Chollet et al., "Keras," GitHub, 2015. [Online]. Available: <https://keras.io>
- [3] G. Bradski, "The OpenCV Library," Dr. Dobb's Journal of Software Tools, 2000.
- [4] Streamlit Inc., "Streamlit — A Faster Way to Build and Share Data Apps," 2019. [Online]. Available: <https://streamlit.io>
- [5] M. Loey, G. Manogaran, and N. E. M. Khalifa, "Fighting Against COVID-19: A Novel Deep Learning Model Based on YOLO-v2 with ResNet-50 for Medical Face Mask Detection," Sustainable Cities and Society, vol. 65, 2021.
- [6] B. Qin and D. Li, "Identifying Facemask-Wearing Condition Using Image Super-Resolution with Classification Network to Prevent COVID-19," Sensors, 2020.
- [7] N. Ejaz et al., "Detection of COVID-19 Face Mask Wearing Using Custom YOLOv3," International Journal of Computer Science and Network Security, 2021.

Appendix

Appendix A: Sample Outputs

This section presents sample outputs of the Face Mask Detection System developed using a CNN model and deployed with Streamlit.

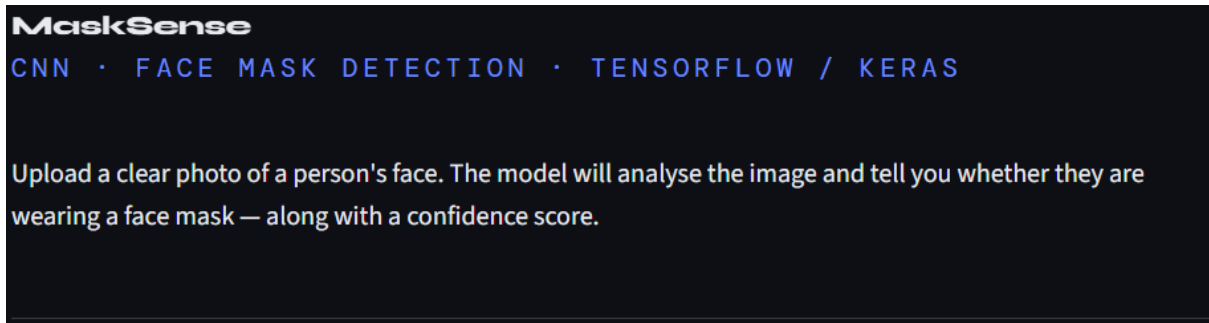


Figure A.1: Introduction Section of the website

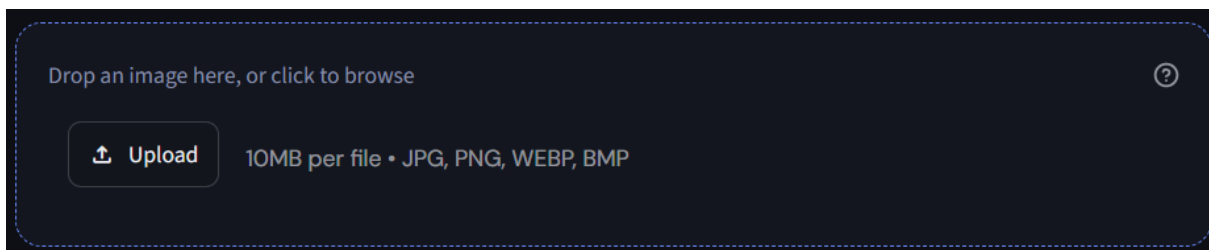


Figure A.2: Image Upload Interface

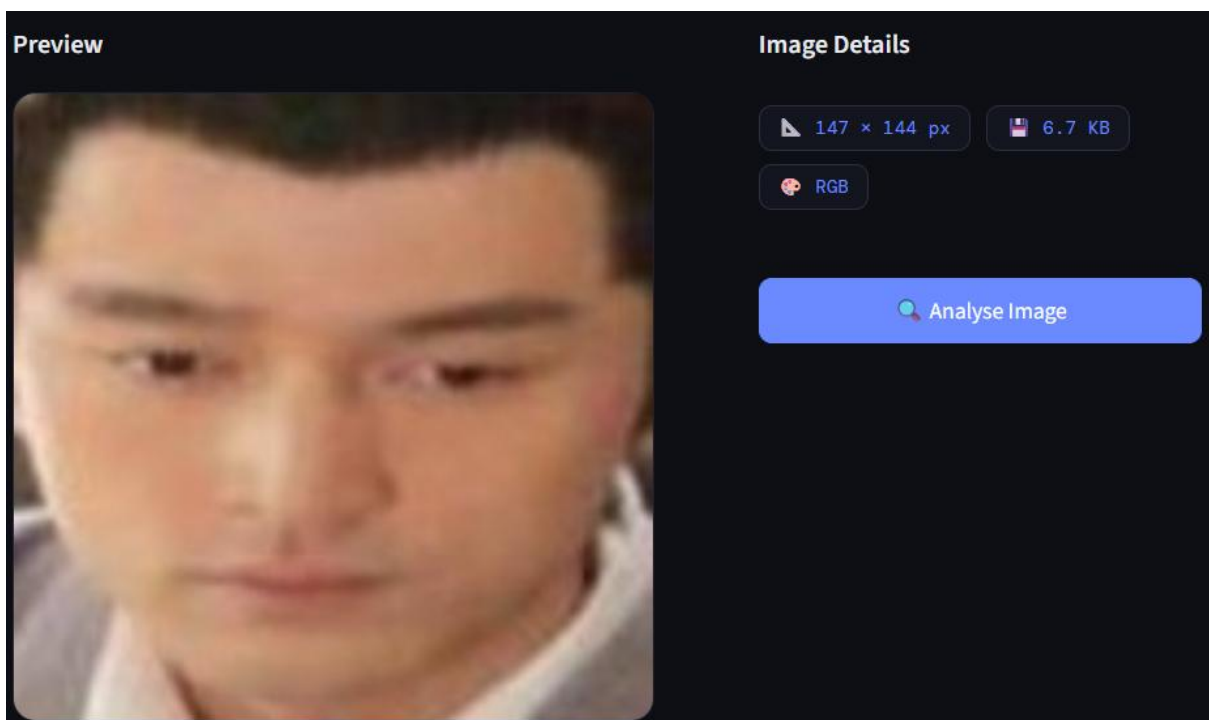


Figure A.3: Image Preview and Details

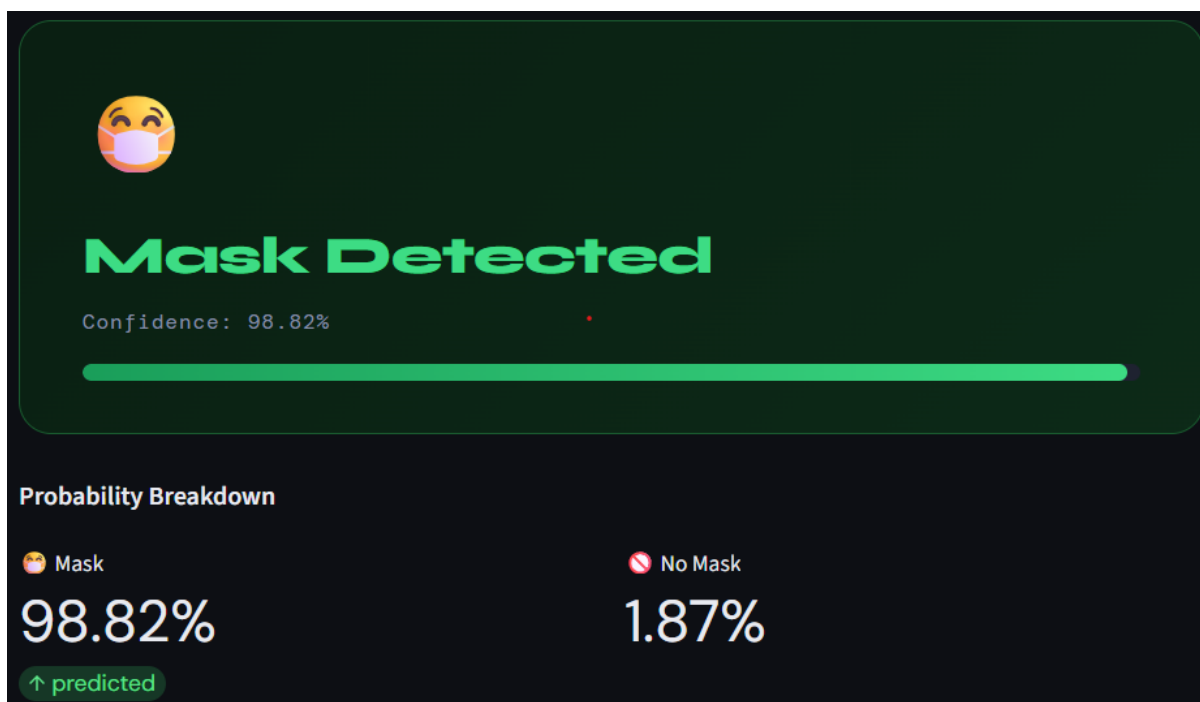


Figure A.4: Final Prediction (Mask Detected)

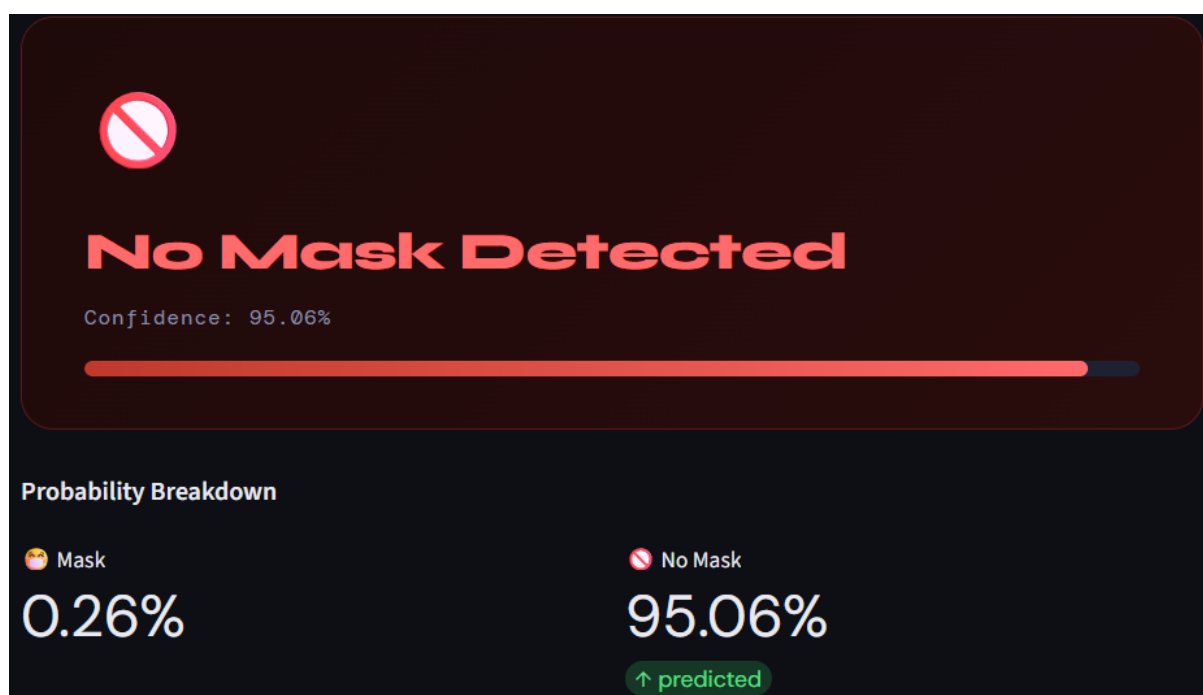


Figure A.5: Final Prediction (Mask Not Detected)